

Rapport Complet du Laboratoire

Analyse Dynamique Android avec MobSF et

DIVA

Incluant Setup, Exploitation et Dépannage

Sécurité Mobile Avancée

Laboratoire Pédagogique

Cadre légal et sécurisé

25 Mars 2024

Contents

1	Prérequis et Configuration du Laboratoire	3
1.1	Configuration matérielle minimale	3
1.2	Logiciels requis	3
1.3	Vérification des prérequis	3
2	Étapes de Setup Détaillées	4
2.1	Étape 1: Création de l'AVD sans Play Store (10 minutes)	4
2.1.1	Procédure complète	4
2.2	Étape 2: Cloner MobSF et scripts officiels (2 minutes)	4
2.3	Étape 3: Lancer l'émulateur avec le script MobSF	5
2.3.1	Sous Linux/macOS	5
2.3.2	Sous Windows	5
2.4	Étape 4: Vérifier la connexion ADB	5
2.5	Étape 5: Installer et lancer MobSF via Docker	7
2.5.1	Pull l'image Docker	7
2.5.2	Lancer MobSF avec Docker	7
2.6	Étape 6: Accéder à MobSF et se connecter	7
2.7	Étape 7: Télécharger l'APK DIVA	8
3	Analyse Statique et Dynamique de DIVA	9
3.1	Étape 1: Upload de l'APK dans MobSF	9
3.2	Résultats de l'analyse statique	9
3.3	Lancer l'analyse dynamique	9
3.4	Lancer DIVA dans l'émulateur	9
4	Menus et Fonctionnalités de l'Analyseur Dynamique	10
4.1	Vue d'ensemble des menus disponibles	10
4.2	Guide d'utilisation des menus	10
4.2.1	1. Logcat Stream (Logs en temps réel)	10
4.2.2	2. Network Traffic Monitor	11
4.2.3	3. File Monitor	11
4.2.4	4. Frida Instrumentation (Hooks Avancés)	11
4.2.5	5. Exported Activity Tester	12
4.2.6	6. TLS/SSL Security Tester	12
5	Les 13 Challenges DIVA Expliqués	13
5.1	Vue d'ensemble	13
5.2	Guide d'exploitation de chaque challenge	13
6	Guide de Dépannage Complet	15
6.1	Problème 1: Dynamic Analysis Failed	15
6.1.1	Symptômes	15
6.1.2	Causes possibles et solutions	15
6.2	Problème 2: Émulateur très lent	15
6.2.1	Symptômes	15
6.2.2	Solutions	15
6.3	Problème 3: Docker ne se connecte pas à l'émulateur	16
6.3.1	Symptômes	16

6.3.2	Solutions	16
6.4	Problème 4: Certificate CA non installé (Proxy HTTPS échoue)	16
6.4.1	Symptômes	16
6.4.2	Solutions	17
6.5	Problème 5: Frida Instrumentation ne fonctionne pas	17
6.5.1	Symptômes	17
6.5.2	Solutions	17
7	Résumé et Conclusion du Laboratoire	19
7.1	Ce que vous avez appris	19
7.2	Techniques maîtrisées	19
7.3	Prochaines étapes	19
7.4	Ressources officielles	19
7.5	Signature et Validation	20

1 Prérequis et Configuration du Laboratoire

1.1 Configuration matérielle minimale

Composant	Requirement
Processeur	64-bit (Intel/AMD) avec virtualisation activée
RAM	Minimum 8 GB (recommandé: 16 GB)
Espace disque	50 GB libres (AVD + Docker + downloads)
Système d'exploitation	Windows 10+, macOS 10.14+, Linux (Ubuntu 18.04+)
Connexion internet	Requise (downloads initiaux: 3-4 GB)

1.2 Logiciels requis

Logiciel	Version	Note
Android Studio	2023.1+	Inclut SDK Platform-Tools et ADB
Docker Desktop	v24.0+	Configuré et en cours d'exécution
Git	2.30+	Pour cloner les repos
Python	3.8+	Généralement pré-installé
JDK	11+	Fourni avec Android Studio

1.3 Vérification des prérequis

Avant de commencer, vérifiez chaque point:

```
# Vérifier Docker
docker --version
# Output: Docker version 24.0.2

# Vérifier ADB
adb version
# Output: Android Debug Bridge version 1.0.41

# Vérifier Git
git --version
# Output: git version 2.41.0

# Vérifier virtualisation (Linux)
grep -o 'vmx\|svm' /proc/cpuinfo | wc -l
# Output: > 0 (si résultat est 0, virtualisation n'est pas activée)

# Vérifier espace disque disponible
df -h /
# Output: doit afficher au moins 50 GB libres
```

2 Étapes de Setup Détaillées

2.1 Étape 1: Création de l'AVD sans Play Store (10 minutes)

2.1.1 Procédure complète

1. Ouvrir Android Studio

- Lancez Android Studio
- Cliquez sur Tools → Device Manager
- Cliquez sur Create Virtual Device

2. Sélectionner le modèle

- Choisissez Pixel 5 (ou Pixel 6)
- Cliquez Next

3. Sélectionner l'image système

- **IMPORTANT:** Choisissez Google APIs **SANS** « Google Play »
- Sélectionnez API 30 (Android 11) ou API 29 (Android 10)
- **CRUCIAL:** Architecture = x86_64
- Cliquez Download si nécessaire (attend 2-3 minutes)
- Cliquez Next

4. Configuration avancée

- RAM: 4096 MB (minimum)
- VM Heap: 512 MB
- Stockage: 2048 MB
- Cliquez Show Advanced Settings → vérifiez les paramètres

5. Nommer l'AVD

- Nom suggéré: MobSF_DIVA_API30
- Cliquez Finish

Vérification: Dans Device Manager, vous voyez l'AVD listé sans Play Store.

2.2 Étape 2: Cloner MobSF et scripts officiels (2 minutes)

```
# Terminal/CMD - Ouvrez une invite de commandes

# Cloner le repo MobSF
git clone https://github.com/MobSF/Mobile-Security-Framework-MobSF.git

# Naviguer dans le dossier
cd Mobile-Security-Framework-MobSF
```

```
# Vérifier les scripts
ls scripts/start_avd.*

# Output (Linux/Mac):
# scripts/start_avd.sh

# Output (Windows):
# scripts\start_avd.ps1
```

Vérification: Vous avez accès aux scripts de démarrage de MobSF.

2.3 Étape 3: Lancer l'émulateur avec le script MobSF

2.3.1 Sous Linux/macOS

```
# Terminal dans le dossier MobSF/
./scripts/start_avd.sh MobSF_DIVA_API30

# Output:
# Available AVDs:
# 1) MobSF_DIVA_API30
# 2) Other AVD...
#
# Select AVD (enter number): 1
# Starting emulator...
# emulator: INFO | Android Virtual Device Version Log
# emulator: Starting QEMU main loop
```

2.3.2 Sous Windows

```
# PowerShell dans le dossier MobSF/
.\scripts\start_avd.ps1 MobSF_DIVA_API30

# Ou manuellement:
emulator -avd MobSF_DIVA_API30 -writable-system
```

Attendre 30-60 secondes que l'émulateur démarre complètement.

2.4 Étape 4: Vérifier la connexion ADB

```
# Nouveau terminal/CMD
adb devices

# Output ATTENDU:
# List of attached devices
# emulator-5554          device
#
# Si vous voyez "device", c'est bon!
# Si vous voyez "offline", attendez 10 secondes et réessayez
# Si aucun device n'apparaît, redémarrez l'émulateur
```

```
# Vérifier l'accès root
adb shell id

# Output ATTENDU (root):
# uid=0(root) gid=0(root)
```

Vérification: Vous voyez `emulator-XXXX` et `uid=0(root)`.

2.5 Étape 5: Installer et lancer MobSF via Docker

2.5.1 Pull l'image Docker

```
# Terminal NOUVEAU (sans fermer l'émulateur!)
docker pull opensecurity/mobile-security-framework-mobsf:latest

# Output:
# latest: Pulling from opensecurity/mobile-security-framework-mobsf
# Digest: sha256:...
# Status: Downloaded newer image for opensecurity/...
```

2.5.2 Lancer MobSF avec Docker

```
# IMPORTANT: Remplacer emulator-5554 par votre identifiant exact!

docker run -it --rm \
  -p 8000:8000 \
  -e MOBSF_ANALYZER_IDENTIFIER=emulator-5554 \
  opensecurity/mobile-security-framework-mobsf:latest

# Output:
# [*] Starting MobSF...
# [*] Loading plugins...
# [*] Starting Frida Server on device...
# [*] Installing certificate...
# [+] MobSF Dynamic Analysis Ready
# [+] Open http://127.0.0.1:8000 in your browser
# Listening on 0.0.0.0:8000
```

Vérification: MobSF écoute sur le port 8000.

2.6 Étape 6: Accéder à MobSF et se connecter

1. Ouvrez votre navigateur web
2. Allez à: <http://127.0.0.1:8000>
3. Login:
 - Username: mobsf
 - Password: mobsf
4. Cliquez Login

Vérification: Vous êtes connecté à MobSF et voyez le dashboard.

2.7 Étape 7: Télécharger l'APK DIVA

```
# Option 1: Via site officiel
# Allez sur: http://www.payatu.com/damn-insecure-and-vulnerable-app/
# Téléchargez: DIVA-debug.apk (ou diva-beta.tar.gz)

# Option 2: Via GitHub (plus rapide)
git clone https://github.com/payatu/diva-android.git
cd diva-android/apks
# Trouvez: DIVA-debug.apk ou diva-beta.tar.gz

# Option 3: Direct download (si disponible)
# wget https://github.com/payatu/diva-android/releases/download/v2.0/DIVA-debug.apk
```

Vérification: Vous avez le fichier DIVA-debug.apk (ou diva.apk).

3 Analyse Statique et Dynamique de DIVA

3.1 Étape 1: Upload de l'APK dans MobSF

1. Dans MobSF, cliquez **Upload & Analyze**
2. Sélectionnez votre fichier `DIVA-debug.apk`
3. Cliquez **Analyze**
4. Attendez 2-3 minutes (analyse statique)

3.2 Résultats de l'analyse statique

Une fois terminée, vous voyez un rapport contenant:

Section	Contenu
App Information	Package name, version, permissions
Manifest Analysis	Activities, services, intent-filters
Binary Analysis	Strings, API calls
Code Analysis	Vulnerabilities détectées
Security Score	Score global (ex: 38/100)

3.3 Lancer l'analyse dynamique

1. Dans le rapport statique, cherchez le bouton **Start Dynamic Analysis**
2. Cliquez dessus
3. MobSF va:
 - Installer DIVA sur l'émulateur
 - Lancer Frida Server
 - Configurer le proxy HTTPS
 - Ouvrir l'interface Dynamic Analyzer
4. Cela prend 1-2 minutes

3.4 Lancer DIVA dans l'émulateur

Une fois Dynamic Analysis prêt:

1. Regardez l'écran mirrored dans MobSF (à droite)
2. Cherchez l'icône DIVA
3. Cliquez sur elle pour lancer l'app
4. DIVA démarre et affiche 13 challenges

4 Menus et Fonctionnalités de l'Analyseur Dynamique

4.1 Vue d'ensemble des menus disponibles

Menu	Fonction	Utilité en analyse dynamique
Stop Screen	Arrêter le mirroring	Arrête l'affichage de l'écran de l'émulateur (libère les ressources)
Remove Root CA	Supprimer le certificat CA MobSF	Nettoie l'émulateur après interception HTTPS et revient à un état plus sûr
Unset HTTP(S) Proxy	Désactiver le proxy réseau	Arrête la redirection du trafic vers MobSF (pour revenir au trafic normal)
TLS/SSL Security Tester	Tester la sécurité TLS/SSL	Vérifie la validation des certificats, l'acceptation de certificats non fiables et les faiblesses réseau
Exported Activity Tester	Tester les activités exportées	Montre si des activités exposées peuvent être lancées depuis d'autres apps
Activity Tester	Tester les activités	Lance et observe les écrans internes pour comprendre le comportement
Get Dependencies	Récupérer les dépendances	Identifie bibliothèques, composants et éléments utiles à l'analyse
Take a Screenshot	Capturer l'écran	Prend une capture d'écran pour documenter une vulnérabilité
Logcat Stream	Afficher les logs en temps réel	Très utile pour détecter les erreurs, exceptions et fuites d'infos
Network Traffic Monitor	Monitor le trafic réseau	Capture et analyse tout le trafic HTTP/HTTPS
Frida Instrumentation	Hooks sur le runtime	Permet d'injecter du code et de hooker des méthodes
File Monitor	Monitor les fichiers créés	Détecte et affiche tous les fichiers créés par l'app
Intent Monitor	Monitor les intents	Capture tous les intents envoyés/reçus
Generate Report	Générer le rapport final	Crée un rapport PDF/JSON avec tous les résultats

4.2 Guide d'utilisation des menus

4.2.1 1. Logcat Stream (Logs en temps réel)

Utilisation :

1. Cliquez sur **"Logcat_Stream"** dans MobSF
2. Interagissez avec DIVA (entrez des données, naviguez)
3. Observez les logs en temps réel qui s'**affichent**

```
Exemple de découverte:
[DEBUG] LoginFragment: Username=admin
[DEBUG] LoginFragment: Password=mysecretpassword123 <- CRITIQUE!
```

4.2.2 2. Network Traffic Monitor

Utilisation:

1. Cliquez sur "NetworkTrafficMonitor"
2. Toutes les requêtes HTTP/HTTPS apparaissent
3. Cliquez sur une requête pour voir les détails
4. Observez les headers, body, response

Découvertes possibles:

- Secrets dans les headers (Authorization, API-Key)
- Données sensibles en plaintext
- HTTP au lieu de HTTPS

4.2.3 3. File Monitor

Utilisation:

1. Cliquez sur "FileMonitor"
2. Naviguez dans DIVA
3. Chaque fichier créé s'affiche

Découvertes possibles:

- Fichiers de credentials en plaintext
- Bases de données non chiffrées
- Fichiers avec permissions publiques

4.2.4 4. Frida Instrumentation (Hooks Avancés)

Utilisation:

1. Cliquez sur "Frida" dans MobSF
2. Cliquez "Spawn & Inject"
3. Écrivez votre script Frida

Exemple 1: Intercepter les logs

```
Java.perform(function() {
    var Log = Java.use("android.util.Log");
    Log.d.overload('java.lang.String', 'java.lang.String').implementation =
    function(tag, msg) {
        console.log "[" + tag + "] " + msg);
        return this.d(tag, msg);
    };
});
```

Exemple 2: Bypass d'authentification

```
Java.perform(function() {
```

```
var AuthMgr = Java.use("jakhar.aspl.diva.AuthManager");
AuthMgr.verifyPassword.implementation = function(p, h) {
    console.log("[BYPASS] Auth bypassed!");
    return true;
};
});
```

Exemple 3: Extraire les secrets

```
Java.perform(function() {
    var ApiConfig = Java.use("jakhar.aspl.diva.ApiConfig");
    console.log("API_KEY = " + ApiConfig.API_KEY.value);
});
```

4.2.5 5. Exported Activity Tester

Utilisation:

1. Cliquez sur **"Exported_Activity_Tester"**
2. MobSF affiche toutes les activités exportées
3. Cliquez sur une activity pour la lancer

Découvertes possibles:

- Activities sans protection
- Activities acceptant des intent extras
- Escalade de privilèges via intent manipulation

4.2.6 6. TLS/SSL Security Tester

Utilisation:

1. Cliquez sur **"TLS/SSL_Security_Tester"**
2. MobSF teste:
 - Validation des certificats
 - Acceptance de certificats non fiables
 - Version TLS utilisée
 - Algorithmes de chiffrement

Résultats:

Si HTTPS stricte: **"TLS_validation:_STRICT"**

Si SSL accepte tout: **"WARNING:_Insecure_SSL_configuration_detected"**

5 Les 13 Challenges DIVA Expliqués

5.1 Vue d'ensemble

DIVA comporte 13 challenges vulnérables couvrant les principales vulnérabilités Android:

#	Challenge	Description et Exploitation
1	Insecure Logging	Mots de passe en plaintext dans Logcat. Exploitation: Ouvrir Logcat Stream et voir les credentials.
2	Hardcoded Secrets	Clés API et tokens en clair dans le code. Exploitation: Hook Frida ou observer le Network Monitor.
3	Insecure Storage	Données sauvegardées sans chiffrement. Exploitation: File Monitor + adb shell pour accéder aux fichiers.
4	Weak Cryptography	Utilisation de DES et MD5 au lieu d'AES/SHA-256. Exploitation: Frida hook sur Cipher.getInstance().
5	Access Control	Activités non protégées accessibles. Exploitation: Exported Activity Tester.
6	Injection (SQL)	Vulnérabilités SQL injection. Exploitation: Passer du code SQL dans les champs.
7	Network Traffic	HTTP plaintext au lieu de HTTPS. Exploitation: Network Traffic Monitor.
8	Intent Data	Intent filters non sécurisés. Exploitation: Intent Monitor ou adb shell am start.
9	Native Code	Fonctions natives (JNI) vulnérables. Exploitation: Reverse engineering avec Frida.
10	Input Validation	Absence de validation des entrées. Exploitation: Injection de caractères spéciaux.
11	Custom Protocol	Protocoles non sécurisés. Exploitation: Network Monitor + Frida.
12	Broken Crypto	Gestion incorrecte des clés. Exploitation: Frida pour extraire les clés en mémoire.
13	Runtime Manip.	Facilité de bypass via Frida/Xposed. Exploitation: Frida hooks pour contourner les protections.

5.2 Guide d'exploitation de chaque challenge

Pour chaque challenge, dans MobSF Dynamic Analyzer:

1. **Interagissez** avec le challenge dans l'écran mirrored
2. **Observez** Logcat Stream, Network Monitor, File Monitor
3. **Documentez** les informations sensibles trouvées
4. **Utilisez** Frida si besoin d'accès plus profond
5. **Capturer** un screenshot de la vulnérabilité

6 Guide de Dépannage Complet

6.1 Problème 1: Dynamic Analysis Failed

6.1.1 Symptômes

Vous cliquez sur Start Dynamic Analysis et obtenez une erreur:

```
Error: Dynamic Analysis Failed
Could not connect to device
```

6.1.2 Causes possibles et solutions

1. Émulateur non lancé

- Vérifiez: `adb devices`
- Attendu: `emulator-5554 device`
- **Solution:** Relancez l'émulateur avec `./scripts/start_avd.sh`

2. MobSF ne voit pas l'émulateur

- Cause: Variable `MOBSF_ANALYZER_IDENTIFIER` incorrecte
- **Solution:** Vérifiez votre identifiant exact avec `adb devices`
- Relancez Docker avec le bon identifiant:

```
adb devices
# Output: emulator-5554 device

# Relancer Docker avec le bon ID
docker run -it --rm -p 8000:8000 \
  -e MOBSF_ANALYZER_IDENTIFIER=emulator-5554 \
  opensecurity/mobile-security-framework-mobsf:latest
```

3. Frida Server ne démarre pas

- Cause: Émulateur trop ancien ou kernel incompatible
- **Solution:** Utilisez API 29 ou 30 x86_64

6.2 Problème 2: Émulateur très lent

6.2.1 Symptômes

L'écran mirrored est lent, ou l'émulateur est laggy.

6.2.2 Solutions

1. Réduire la résolution dans l'émulateur

- Android Studio → AVD → Edit → Screen: 720x1280

2. Augmenter la RAM allouée

- Minimum 4GB, idéal 6-8GB
3. **Utiliser API 29 au lieu d'API 31+**
 - API 29 est plus rapide que les versions plus récentes
 4. **Arrêter les processus en arrière-plan**
 - Fermez Chrome, Slack, etc.
 - Libérez au moins 4GB de RAM libre
 5. **Activer GPU Acceleration**
 - Android Studio → AVD → Edit → Advanced Settings → GPU: Hardware

6.3 Problème 3: Docker ne se connecte pas à l'émulateur

6.3.1 Symptômes

MobSF démarre mais n'arrive pas à voir l'émulateur.

6.3.2 Solutions

1. **Sous Linux: Ajouter `--net=host`**

```
docker run -it --rm \
  --net=host \
  -p 8000:8000 \
  -e MOBSF_ANALYZER_IDENTIFIER=emulator-5554 \
  opensecurity/mobile-security-framework-mobsf:latest
```

2. **Sous Windows/Mac: Vérifier l'IP**

```
# Trouver l'IP du Docker host
docker inspect <container_id> | grep "IPAddress"

# Utiliser cette IP au lieu de localhost si besoin
```

3. **Redémarrer Docker Desktop**

- Fermez Docker Desktop complètement
- Attendez 10 secondes
- Relancez Docker Desktop
- Attendez que le logo se stabilise

6.4 Problème 4: Certificate CA non installé (Proxy HTTPS échoue)

6.4.1 Symptômes

HTTPS traffic ne s'affiche pas dans Network Monitor.

6.4.2 Solutions

1. Vérifier que le CA est installé

```
# Dans l'émulateur
adb shell su -c "ls_/system/etc/security/cacerts/" | grep mobsf

# Si vide, relancer MobSF et cliquer "Install Certificate"
```

2. Réinstaller le certificat

- Dans MobSF Dynamic Analyzer
- Cliquez: Tools → Install Root CA
- Attendez 30 secondes

3. Comme dernier recours: Nettoyer et relancer

```
# Arrêter MobSF (Ctrl+C)
# Arrêter l'émulateur
adb emu kill

# Relancer
./scripts/start_avd.sh MobSF_DIVA_API30
# Puis Docker
docker run -it --rm -p 8000:8000 ...
```

6.5 Problème 5: Frida Instrumentation ne fonctionne pas

6.5.1 Symptômes

Les hooks Frida ne s'exécutent pas, ou "Frida Server not responding".

6.5.2 Solutions

1. Vérifier que Frida Server tourne

```
# Dans l'émulateur
adb shell ps | grep frida

# Expected output:
# root ... /data/local/tmp/frida-server
```

2. Si Frida n'est pas lancé, relancer MobSF

- Cliquez dans MobSF: Tools → Restart Frida

3. Vérifier la syntaxe JavaScript du hook

```
// Syntaxe correcte
Java.perform(function() {
    console.log("Hello from Frida!");
});
```

```
//  Syntaxe incorrecte
Java.perform(function() {
    console.log("Missing semicolon")  // <- Missing ;
});
```

4. Utiliser console.log pour debugger

```
Java.perform(function() {
    try {
        var MyClass = Java.use("com.example.MyClass");
        console.log("[OK] Class loaded");
    } catch(e) {
        console.log("[ERROR] " + e);
    }
});
```

7 Résumé et Conclusion du Laboratoire

7.1 Ce que vous avez appris

1. **Configuration d'un émulateur Android propre** sans Play Store
2. **Déploiement de MobSF** via Docker
3. **Analyse dynamique complète** (runtime execution)
4. **Interception de trafic HTTPS** via un proxy global
5. **Instrumentation avancée** avec Frida et hooks
6. **Détection de vulnérabilités** en temps réel
7. **Exploitation des 13 challenges DIVA**

7.2 Techniques maîtrisées

- Logcat Stream → détection logs sensibles
- Network Monitor → interception HTTP/HTTPS
- File Monitor → détection stockage insécurisé
- Intent Monitor → analyse IPC
- Frida Hooks → instrumentation runtime
- Activity Tester → test des composants exportés
- TLS/SSL Tester → validation certificats

7.3 Prochaines étapes

Vous pouvez maintenant:

1. Analyser vos propres applications Android
2. Répéter le lab avec d'autres APKs vulnérables (InsecureBankv2, AndroGoat)
3. Écrire vos propres hooks Frida avancés
4. Combiner l'analyse statique et dynamique
5. Contribuer à des projets open-source de sécurité Android

7.4 Ressources officielles

- **MobSF Documentation:** <https://github.com/MobSF/docs>
- **DIVA Challenges:** <https://github.com/payatu/diva-android>
- **Frida Documentation:** <https://frida.re/docs/>
- **OWASP MASTG:** <https://mas.owasp.org/MASTG/>
- **Android Security Guide:** <https://source.android.com/docs/security>

7.5 Signature et Validation

Étudiant(e)	Date	Signature
[Nom]	25/03/2024	

Document: Rapport Complet Laboratoire Analyse Dynamique

Outils: MobSF v4.0.3, Frida 16.1.2, Docker v24.0

Application: DIVA v2.0 (13 Challenges)

Émulateur: AVD Pixel 5 API 30 (x86_64)

Version du rapport: 2.0 (Ultra-Complet)

Status: TOUS LES ÉLÉMENTS REMPLIS